

```
import os
import sys
import time
import logging
from datetime import datetime
```

Configure instant system logs for Render console tracking

```
logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s [%(levelname)s] %
(message)s',
    handlers=[logging.StreamHandler(sys.stdout)]
)
```

```
class EnterpriseAutonomousPlatform:
    def init(self):
        self.brand = "18&ok LLC"
```

```
self.stripe_key =  
os.getenv("STRIPE_SECRET_KEY")  
self.twilio_sid =  
os.getenv("TWILIO_ACCOUNT_SID")  
self.twilio_token =  
os.getenv("TWILIO_AUTH_TOKEN")  
self.owner_phone =  
os.getenv("OWNER_CELL_PHONE")
```

```
# Internal accounting simulations
```

```
self.liquidity = 50000.00  
self.total_yield = 0.0
```

```
def verify_credentials(self):
```

```
    """Ensures secure tokens are injected before  
    starting operations."""
```

```
    keys = {  
        "STRIPE_SECRET_KEY": self.stripe_key,  
        "TWILIO_ACCOUNT_SID": self.twilio_sid,  
        "TWILIO_AUTH_TOKEN": self.twilio_token,
```

```
"OWNER_CELL_PHONE": self.owner_phone
}
missing = [k for k, v in keys.items() if v is None]
if missing:
    logging.error(f"Initialization Failed. Missing
secure environment variables: {', '.join(missing)}")
return False
return True
```

```
def run_gear_agent(self):
logging.info("[GEAR AGENT] Initiating B2B
liquidation market scan...")
# Production API integration point
deal_cost = 4000
resale_value = 9500
if self.liquidity >= deal_cost:
    self.liquidity -= deal_cost
    net = resale_value - deal_cost
    self.total_yield += net
    self.liquidity += resale_value
```

```
self.send_sms(f" 18&ok Alert: Gear Agent  
completed automated flip. Net Profit Collected:  
${net:,.2f}")
```

```
def run_property_agent(self):  
logging.info("[PROPERTY AGENT] Scanning  
county tax lien databases...")  
# Production GIS data connection point  
lien_cost = 10000  
market_value = 35000  
if self.liquidity >= lien_cost:  
    self.liquidity -= lien_cost  
    equity = market_value - lien_cost  
    self.total_yield += equity  
    self.liquidity += market_value  
    self.send_sms(f" 18&ok Alert: Property Agent  
assigned distressed parcel. Equity Added: ` $  
{equity:,.2f}")
```

```
def run_tech_agent(self):
```

[logging.info](#)("[TECH AGENT] Evaluating micro-SaaS digital software assets...")

Production acquisition repository point

buyout = 3000

revenue_yield = 8000

if self.liquidity >= buyout:

self.liquidity -= buyout

self.total_yield += revenue_yield

self.liquidity += revenue_yield

self.send_sms(f" ⚡ 18&ok Alert: Tech Agent
secured SaaS recurring cash flow. Added:
\${revenue_yield:,.2f}")

def send_sms(self, message):

[logging.info](#)(f"[SMS DISPATCH] Route ->
{self.owner_phone}: {message}")

Programmatic REST execution framework:

import requests

url = f"<https://twilio.com>{self.twilio_sid}/
Messages.json"

```
# payload = {"From": "+185555550100", "To":  
self.owner_phone, "Body": message}  
# requests.post(url, auth=(self.twilio_sid,  
self.twilio_token), data=payload)
```

```
def start_perpetual_loop(self):  
    """The core runtime clock that forces the  
    platform to run 24/7 on autopilot."""  
    logging.info(f"=== {self.brand.upper()} FULLY  
HANDS-OFF PLATFORM ACTIVATED ===")
```

```
while True:
```

```
try:
```

```
logging.info(f"Automated sweeps refreshed at  
{datetime.now().strftime('%Y-%m-%d %H:%M:  
%S')}}")
```

```
self.run_gear_agent()
```

```
self.run_property_agent()
```

```
self.run_tech_agent()
```

```
logging.info(f"Current Loop Complete. Available  
Cash: `${self.liquidity:,.2f}` | Cumulative Profit: $  
{self.total_yield:,.2f}")
```

```
logging.info("System entering 1-hour sleep  
cycle...")
```

```
time.sleep(3600) # Wait exactly 60 minutes
```

```
except Exception as e:
```

```
logging.error(f"System anomaly intercepted:  
{str(e)}")
```

```
time.sleep(60) # Safe delay before automatic  
retry loop
```

```
if name == "main":
```

```
platform = EnterpriseAutonomousPlatform()
```

```
if platform.verify_credentials():
```

```
platform.start_perpetual_loop()
```